

Project Documentation: Parameterized Arithmetic Logic Unit (ALU)

Digital Design Lab

Project Overview

This lab implements a parameterized **Arithmetic Logic Unit (ALU)** in Verilog HDL [?]. The ALU evaluates 8-bit operands and executes arithmetic, bitwise, or pass-through operations based on a 3-bit operation code (**opcode**) [?]. Additionally, it generates a real-time status output signal (**a_is_zero**) indicating whether operand **in_a** is zero [?].

Specifications & Functionality

- Default Bus Width: `WIDTH = 8 bits` (Parameterized) [?].
- Opcode Width: 3-bit selection vector (`opcode[2:0]`) [?].
- Zero Detection: `a_is_zero` evaluates asynchronously to `1'b1` whenever `in_a == 0`, and `1'b0` otherwise [?].

Opcode Functional Mapping

Opcode Encoding	Instruction Name	Operation	Output Expression
3'b000	HLT	PASS A	$alu_out = in_a$ [?]
3'b001	SKZ	PASS A	$alu_out = in_a$ [?]
3'b010	ADD	Addition	$alu_out = in_a + in_b$ [?]
3'b011	AND	Bitwise AND	$alu_out = in_a \& in_b$ [?]
3'b100	XOR	Bitwise XOR	$alu_out = in_a \wedge in_b$ [?]
3'b101	LDA	PASS B	$alu_out = in_b$ [?]
3'b110	STO	PASS A	$alu_out = in_a$ [?]
3'b111	JMP	PASS A	$alu_out = in_a$ [?]

Table 1: ALU Instruction Set Specifications

Port Summary

Port Name	Direction	Type	Width	Description
in_a	Input	wire	WIDTH	Primary operand vector [?, ?, ?]
in_b	Input	wire	WIDTH	Secondary operand vector [?, ?, ?]
opcode	Input	wire	3	Operation code selector [?, ?, ?]
alu_out	Output	reg	WIDTH	Execution result output [?, ?, ?]
a_is_zero	Output	reg / wire	1	Zero flag for operand A [?, ?, ?]

Table 2: ALU Module Interface

Critical Design Fixes & Analysis

Identified Code Errors & Fixes

An analysis of the initial implementation snippet (alu.v) revealed several critical functional errors [?]:

1. Undeclared Identifier (w): The initial design referenced [w-1:0] while the parameter was declared as parameter WIDTH = 8 [?].
2. Incorrect Opcode Port Declaration: Parameter v=3 with [v:0] created a 4-bit bus ([3:0]), causing port size mismatches with the 3-bit signal in the testbench [?, ?].
3. Extraneous Interface Signal (en): The original code included an en input pin [?]. Neither the testbench [?] nor the specifications [?] include an enable signal.
4. Incomplete Assignment Latch Bug: a_is_zero was only assigned inside a incomplete case(in_a == 0) statement without a default condition [?], causing synthesis tools to infer unwanted transparent latches.

Verilog Source Code & Line-by-Line Explanations

Corrected Design Module (alu.v)

Below is the fully corrected implementation for alu.v:

```

module alu #(
    parameter WIDTH = 8
)(
    input  wire [WIDTH-1:0] in_a,
    input  wire [WIDTH-1:0] in_b,
    input  wire [2:0]      opcode,
    output reg  [WIDTH-1:0] alu_out,
    output wire             a_is_zero
);

    // Continuous assignment for zero flag evaluation
    assign a_is_zero = (in_a == {WIDTH{1'b0}});

    // Combinational block for execution operations
    always @(*) begin
        case (opcode)

```

```

        3'b000: alu_out = in_a;          // HLT: PASS A
        3'b001: alu_out = in_a;          // SKZ: PASS A
        3'b010: alu_out = in_a + in_b;    // ADD
        3'b011: alu_out = in_a & in_b;    // AND
        3'b100: alu_out = in_a ^ in_b;    // XOR
        3'b101: alu_out = in_b;          // LDA: PASS B
        3'b110: alu_out = in_a;          // STO: PASS A
        3'b111: alu_out = in_a;          // JMP: PASS A
        default: alu_out = {WIDTH{1'b0}};
    endcase
end
endmodule

```

4.1.1 Design Code Breakdown

- parameter WIDTH = 8: Dynamic module parameter defining bus width across input/output vectors [?, ?].
- assign a_is_zero = (in_a == 0): Evaluates dynamically to 1'b1 when all bits of in_a are zero, avoiding memory latch generation [?].
- always @(*): Explicit combinational logic block triggering whenever in_a, in_b, or opcode changes value.
- case (opcode): Standard 3-bit operation decoder executing arithmetic/logical functions based on the specification table [?, ?].

Testbench Module (alu_test.v)

```

`timescale 1ns / 1ps

module alu_test;

    localparam WIDTH = 8;
    localparam PASS0=0, PASS1=1, ADD=2, AND=3, XOR=4, PASSB=5, PASS6=6,
        PASS7=7;

    reg [2:0] opcode;
    reg [WIDTH-1:0] in_a;
    reg [WIDTH-1:0] in_b;
    wire a_is_zero;
    wire [WIDTH-1:0] alu_out;

    // Instantiate Unit Under Test (UUT)
    alu #(
        .WIDTH (WIDTH)
    ) alu_inst (
        .opcode (opcode),
        .in_a (in_a),
        .in_b (in_b),
        .a_is_zero (a_is_zero),
        .alu_out (alu_out)
    );

```

```

// Verification Task
task expect;
    input          exp_zero;
    input [WIDTH-1:0] exp_out;
    begin
        if (a_is_zero != exp_zero || alu_out != exp_out) begin
            $display("TEST FAILED");
            $display("At time %0d opcode=%b in_a=%b in_b=%b
                a_is_zero=%b alu_out=%b",
                $time, opcode, in_a, in_b, a_is_zero, alu_out)
            ;
            if (a_is_zero != exp_zero)
                $display("a_is_zero should be %b", exp_zero);
            if (alu_out != exp_out)
                $display("alu_out should be %b", exp_out);
            $finish;
        end else begin
            $display("At time %0d opcode=%b in_a=%b in_b=%b
                a_is_zero=%b alu_out=%b",
                $time, opcode, in_a, in_b, a_is_zero, alu_out)
            ;
        end
    end
endtask

// Test Sequence Execution
initial begin
    opcode=PASS0; in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h42);
    opcode=PASS1; in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h42);
    opcode=ADD;   in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'hC8);
    opcode=AND;   in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h02);
    opcode=XOR;   in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'hC4);
    opcode=PASSB; in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h86);
    opcode=PASS6; in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h42);
    opcode=PASS7; in_a=8'h42; in_b=8'h86; #1 expect(1'b0, 8'h42);
    opcode=PASS7; in_a=8'h00; in_b=8'h86; #1 expect(1'b1, 8'h00);
    $display("TEST PASSED");
    $finish;
end

endmodule

```

Simulation & Verification

Execution Command

To run batch simulation with VCS or Cadence Xcelium [?]:

```
vcs alu.v alu_test.v -R
```

Expected Simulation Output

Upon executing the testbench with the fixed ALU module, the simulation matches expected results [?]:

```
At time 1 opcode=000 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=01000010
At time 2 opcode=001 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=01000010
At time 3 opcode=010 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=11001000
At time 4 opcode=011 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=00000010
At time 5 opcode=100 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=11000100
At time 6 opcode=101 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=10000110
At time 7 opcode=110 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=01000010
At time 8 opcode=111 in_a=01000010 in_b=10000110 a_is_zero=0 alu_out
=01000010
At time 9 opcode=111 in_a=00000000 in_b=10000110 a_is_zero=1 alu_out
=00000000
TEST PASSED
```